

Team Name: lvbel c++

CMP2204 Introduction to Computer Networks – Term Project Report

1. Group Members & Division of Workload The project was developed collaboratively with a clear division of tasks to efficiently implement the required P2P file-sharing protocols.

- **Feyzi Efe Şahin:** Responsible for the core architectural design of the P2P network. Implemented the UDP broadcasting and discovery mechanisms (Chunk Announcer and Chunk Discovery processes), designed the end-user CLI interactions, and handled the unencrypted TCP file transfer (download/upload) logic.
- **Anıl Özçelik:** Responsible for the secure communication layer and logging mechanisms. Implemented the Diffie-Hellman Key Exchange algorithm, integrated the pyDes library for chunk encryption/decryption, handled the generation of upload/download logs with timestamps, and conducted the Wireshark network traffic verification.

2. Development Platform

- **Operating System:** Windows
- **Programming Language:** Python 3.9+
- **Libraries Used:** socket, threading, json, base64, os, time, random, pyDes
- **Network Analysis Tool:** Wireshark

3. Faced Challenges and Solutions During the development and testing phases of the P2P file-sharing application, our team encountered and resolved several technical challenges to meet the project requirements:

- **Challenge 1: LAN Discovery and UDP Broadcast Blocking**
 - *Issue:* Initially, the Chunk Announcer and Chunk Discovery processes failed to detect peers in the Local Area Network because Windows Defender Firewall blocked incoming UDP broadcast packets on port 6000.
 - *Solution:* We resolved this by explicitly configuring inbound rules in the firewall for Python executable binaries and ensuring both testing machines were connected to the same subnet with the network profile set to "Private".
- **Challenge 2: Binary Data Transmission in JSON Payloads**

- *Issue:* Sending raw image bytes directly through TCP sockets alongside text-based metadata (such as chunk names and usernames) caused severe utf-8 encoding errors.
- *Solution:* We implemented base64 encoding. All raw binary data and DES-encrypted bytes were encoded into Base64 strings before being packed into the JSON dictionaries, ensuring safe transmission.
- **Challenge 3: Decryption Padding and File Corruption**
 - *Issue:* The DES encryption algorithm adds padding bytes to the chunk data. Upon decryption, these extra bytes corrupted the final merged files, preventing them from opening properly.
 - *Solution:* We added an "original_size" parameter to the secure JSON payload. The Chunk Downloader process uses this integer to truncate the decrypted byte array back to its exact original length, eliminating data corruption.
- **Challenge 4: Plaintext Name Visibility in Wireshark**
 - *Issue:* The assignment required demonstrating team names in plaintext during an unencrypted image chunk exchange. Since image data is naturally unreadable, our names were not visible in the TCP stream.
 - *Solution:* We modified the JSON request and response dictionaries to explicitly include "sender_name" and "requester_name" parameters. This allowed Wireshark's ASCII viewer to clearly display our names in plaintext during unencrypted transfers.